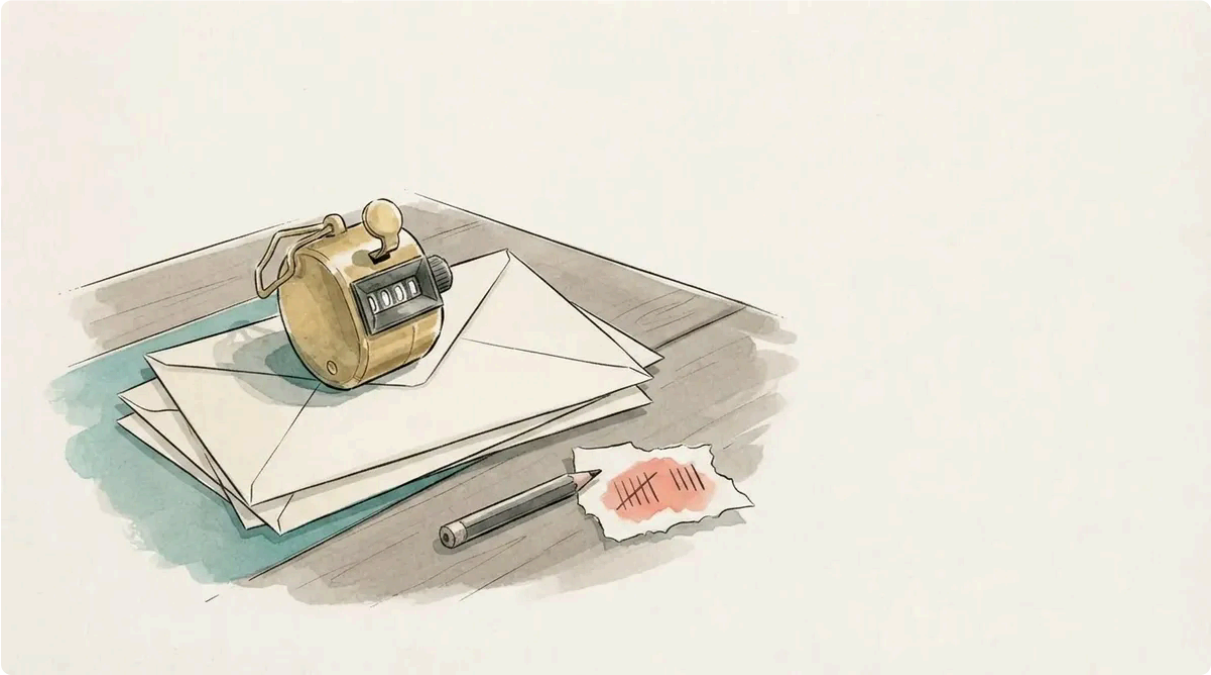
Reader analytics, kept to myself

Emil Lindfors | September 03, 2026



Six of every seven events my new analytics collected in their first hours were timings of the same fifteen files. The stylesheet, two fonts, the search index, a hero image, measured again on every page view, sixty fields each. Their URLs carry a content hash, so the measurement can never change. 82% of the volume, and not one number in it I would ever look at.

That setting is off now, and the rest of this post is the reasoning around it: why a blog that refuses to track its newsletter readers tracks its web readers at all, what I decided not to collect and why, which tools I looked at before settling on the one I already had, and what a day of real data taught me about clicks on a static site.

Why track anything

Metrics and analytics is how one gathers feedback for how well something works.

That is the whole argument, and it applies to writing as much as to a data pipeline. I publish a post and then hear nothing. A newsletter reader might reply, a colleague might mention it, but the ordinary case is silence, and silence is a bad signal to write against. I want to know whether anyone reads the long ones to the end, whether the series pages get used, whether the PDF button is a feature or a decoration.

There is of course a dark side when one optimizes only for things that gather clicks, but if done in a non-hyperoptimized way it's good to know if people like what you write, and it keeps you engaged as a writer as well to write more content, so we want to have a virtuous cycle of creativity.

The dark side has a literature of its own. Coad, Nightingale, Stilgoe and Vezzani (2020) opened a special issue on it by pointing out that innovation studies had spent decades assuming innovation is good and measuring how to get more of it, while the harms, from addictive products to the attention economy, were studied by other people in other journals. Analytics on a website is a small, exact example. The moment you can see which headline got clicks, the headlines start bending towards clicks. The writer who wanted feedback has become a writer optimising a number. So the design problem is to get the feedback without building the incentive, and that is a problem with a framework.

Responsible innovation, applied to a blog

Stilgoe, Owen and Macnaghten (2013) wrote the framework for a much larger case, the governance of geoengineering research, and it has been used for AI and biotech since. It asks four things of anyone building something whose effects fall on other people:

- **Anticipation.** Think through what the thing could do before it does it. Not prediction, but the discipline of asking "what if" while changes are still cheap.
- **Reflexivity.** Look at your own motives and assumptions as part of the system, since the builder's incentives are inside the design.
- **Inclusion.** Bring in the people affected, and bring them in early enough that they can change something.
- **Responsiveness.** Keep the ability to change course when the first three turn something up.

It was written for scientists and regulators, and a blog is a small thing to point it at, so take the transfer as a checklist and not as a finding. But it reads directly onto the decisions below, and every one of them was easier to make with the four words in front of me.

Anticipation is deciding what not to collect before the first byte arrives. Session replay is off; it would record the screen. Resource timing went off once the 82% showed what it cost. Nothing identifies a user, because there is no login and every id would be invented. The one input field on this site takes an email address, so interaction events mask user input, since they otherwise carry the text of whatever was acted on.

Reflexivity is the paragraph above about headlines, and one more thing. The SDK writes one first-party cookie, `_oo_s`, a session id that follows the reader between pages for the length of a visit. It is not a tracking cookie in the cross-site sense, and nothing is sold, but it is a cookie that exists only to measure, and a cookie like that needs the reader's yes. The first draft of this post admitted to having no banner and argued that the data staying on my machine made up for it. It did not. The bar at the bottom of this page is the correction.

Inclusion is the hard one for a blog. I cannot ask readers before they arrive, but I can ask before I measure. The bar says what is collected and where it goes, *Allow* and *No thanks* are the same size, and until one of them is pressed the analytics script is not even downloaded. *No thanks* is remembered as well as *Allow*, and the *Analytics* link in the footer reopens the choice. Most readers will press nothing, so what I see is a sample of the people who agreed, on top of the sampling described below. For the question I have, whether a post got read, that is enough.

That sample turned out to be small. Two days after the bar went up, the stream held four readers who were not me, on a day I had spent posting the swarm piece everywhere I could. So since 5 September the loader sends one thing before it asks: a ping with the page, the referrer and whether the bar has been answered before, and nothing else. No cookie, no storage, no id, so no two pings can be tied to one reader. It is the same class of record as a line in a server log, which a site on a static host does not otherwise have. It exists to be the denominator. Consented views divided by pings is the share of visits the analytics actually describe, and a second event when a button is pressed says how the bar is answered. OpenObserve stores the sender's address on every row it ingests, and for someone who never agreed that is the one field that should not be kept, so a pipeline on the stream drops it for these rows. The text on the bar says a view is counted either way.

Responsiveness is the knob. Everything above is a setting, and the first thing the data did was change one of them.

Keeping it

I don't want to hand the data out to someone else, I want to keep it to myself.

The usual way to get analytics is to paste a snippet from a hosted service, and the service gets a copy of every page view from every visitor, forever, on terms you did not write. The reader did not choose that service; I did, on their behalf. For a site that runs its own mail server for the same reason, that would have been a strange place to stop.

I want to be a good citizen, I suppose, and contribute to values I also share, like not having one's data commoditized and sold.

Sovereignty over your own data is the same question on the model side, where I wrote about it from the customer's chair. Here I am the vendor, and the readers are the customer, and the answer is the same: the data lives on a box I control, in a format I can query, and I can delete all of it with one command.

The alternatives

It's not that hard to set up tracking and not rely on some hosted solution where you don't have control.

What I looked at, as of September 2026:

Tool	Written in	Self-hosted	What it measures
Cloudflare Web Analytics	hosted only	no	page views, web vitals, their beacon
Plausible	Elixir	Community Edition	page views, referrers, goals
Umami	JavaScript	yes	page views, events
GoatCounter	Go	yes	page views, referrers
Matomo	PHP	yes	everything, Google-Analytics-shaped
PostHog	Python and TypeScript	yes, heavy	product analytics, replay, feature flags
OpenObserve	Rust	yes	logs, metrics, traces, and RUM: views, web vitals, errors, actions

Plausible and GoatCounter are the honest small options, and if you want a page-view counter and nothing else, take one of them and stop reading. I wanted more than a counter: web vitals, so I know when a post with three tables and a KaTeX block got slow on phones, and JavaScript errors, because a static site with a citation modal and a search box still has code that can break.

I like OpenObserve as it's written in Rust and has RUM.

And it was already running. The same OpenObserve takes the host's logs and the LLM usage telemetry from my coding agents, so the reader analytics are one more stream in a store I already query, on a box I already back up. RUM, real user monitoring, is the name for the browser side of it: a small script in the page reports what the browser saw. OpenObserve's browser SDK is a fork of Datadog's, which is why the API, the cookie and the event shapes look the way they do.

What the loader does before the SDK exists

The SDK is 178 KB, 61 KB over the wire. The rest of this site's JavaScript is about 16 KB. So the SDK does not go in front of the page, and the whole design of the integration is about when it loads.

The page loads only an 8 KB loader. It sends the ping, then does nothing until the reader has pressed Allow, and then three things:

First, it samples. The roll happens before the SDK is fetched, so an unsampled visit costs nothing at all. The SDK's own sample rate is then pinned at 100, because two rates multiply.

```
var rate = parseInt(cfg.sessionSampleRate, 10);
var seen = sessionStorage.getItem('oo-rum-sampled');
if (seen === null) {
    sampled = Math.random() * 100 < rate;
    sessionStorage.setItem('oo-rum-sampled', sampled ? '1' : '0');
}
if (!sampled) return;
```

Second, it waits. Sampled visits get the SDK after `load`, on an idle callback capped at two seconds. This costs less than it looks, because the SDK registers its performance observers with `buffered: true`, so first paint, largest contentful paint and navigation timing are read back out of the browser's buffer whenever it starts. What a late start does lose is errors thrown before it, so the loader traps those and replays them once the SDK is up.

Third, it reads its configuration from `data-` attributes on its own script tag, written from the site config at build time. Not from an inline script, because the content security policy has no `unsafe-inline` and would drop a config object silently. The client token in there is a write-only ingest key, public by design.

The bundle is vendored into the repo with a pinned checksum, the same choice the fonts made. There is no `package.json` here, Cloudflare Pages runs no install step, and an ordinary page load fetches nothing from a third party.

Two things the first day taught me

Resource timing is the volume. That is the 82% from the top, and I found it on the first evening in the stream's field counts: one record per stylesheet, font, image and script per view, with the SDK even timing its own download. The view events keep first paint, LCP and a resource *count* without it, so it went off, and it goes back on for a week when something specific looks slow.

Clicks on links never arrive. This one cost me an afternoon. The SDK tracks user interactions, so I expected to see which links readers follow. Views arrived, errors arrived, the theme toggle arrived, and not one internal link did. I pointed a headless Chrome at the live site over the DevTools protocol and logged every request to the ingest. The SDK turns a click into an action only after page activity settles, and on a static site a click on a link navigates, which destroys the page first. The beacon sent at unload carried the view update and nothing else. Locally, against a copy of the site on localhost, the same click sometimes made it. Not something to build on.

So the loader records link clicks itself, as a custom action, which is written at once and rides the unload beacon:

```
document.addEventListener('click', function (e) {
    var a = e.target.closest('a[href]');
    if (!a || e.button !== 0) return;
    var url = new URL(a.href, location.href);
    window.OO_RUM.addAction('link', {
        href: url.href,
        internal: url.origin === location.origin,
        text: a.textContent.trim().slice(0, 80)
    });
}, true);
```

Outbound links included, which the SDK would have lost too. The same afternoon explained a second gap: OpenObserve's Sessions page, the one that lists each visit as a path through the site, is hidden unless session replay is on. Replay is off here on purpose. The paths are in the data regardless, one `view` row per page joined by a session id, and this is the query that shows them:

```
SELECT session_id, min(_timestamp) AS started, count(*) AS pages,
       array_agg(view_url ORDER BY _timestamp) AS path
FROM "_rumdata"
WHERE type = 'view'
GROUP BY session_id
ORDER BY started DESC;
```

That is the list I wanted in the first place, without recording anyone's screen.

What I'd tell you

Decide what not to collect before you collect anything. Replay, resource timing, identity. Each is one setting, and each is much easier to leave off than to turn off after you have a dashboard built on it.

Sample in the loader, not in the SDK. Sampling inside the SDK saves ingest. Sampling before you fetch it saves the download for everyone you exclude, which on a text site is the only cost that matters.

Look at the volume on day one. Mine was six of every seven events measuring the same files. You will have your own version.

Do not trust click tracking on a static site. Test it with a real browser and watch the beacon. If the link navigates, record the click yourself.

Ask before the SDK exists. If your loader already decides whether to fetch the SDK, the consent check is twenty lines in front of the sampling roll, and a no costs zero bytes. A

banner bolted on after the script has loaded is a banner about a cookie that is already there.

What comes next is the newsletter. I said in that post that it has no open or click tracking and that I did not want any, and I have changed my mind on half of it. I do want to know whether an issue gets read, because that is the feedback the writing needs. I do not want to know it per reader. The plan is a parameter on the links in each issue, so a visit from the newsletter shows up in the view rows above as coming from that issue and nothing more. Per issue, not per person, is where the line goes, and the four words above are why it goes there.

References

- Coad, A., Nightingale, P., Stilgoe, J., & Vezzani, A. "Editorial: the dark side of innovation". *Industry and Innovation*, vol. 28, no. 1, pp. 102-112, 2020. doi:10.1080/13662716.2020.1818555
- Stilgoe, J., Owen, R., & Macnaghten, P. "Developing a framework for responsible innovation". *Research Policy*, vol. 42, no. 9, pp. 1568-1580, 2013. doi:10.1016/j.respol.2013.05.008